
—

Rapport de projet SAE, résolution du problème de l'anneau étoile

—



Discipline : SAE Optimisation
Encadrants : Lucas Letocart, Mustafa Nabil, Pierre Fouilhoux

Engagement de non-plagiat

Je, soussigné Sabri Rebaine, étudiant en deuxième année d'école d'ingénieur à Sup Galilée, déclare être pleinement conscient que la copie de tout ou partie d'un document, quel qu'il soit, publié sur tout support existant, y compris sur Internet, constitue une violation du droit d'auteur ainsi qu'une fraude caractérisée, tout comme l'utilisation d'outils d'Intelligence Artificielle pour générer une partie de ce rapport ou du code associé.

En conséquence, je déclare que ce travail ne comporte aucun plagiat, et assure avoir cité explicitement, à chaque fois que j'en ai fait usage, toutes les sources utilisées pour le rédiger.

Fait à Sceaux , le 26 Janvier 2026

Signature : Sabri Rebaine

A handwritten signature in black ink, consisting of a stylized, cursive 'S' followed by a long horizontal line extending to the right.

Sommaire

Introduction.....	5
I - Problème de l'anneau étoile, un problème NP-difficile.....	5
II - Résolution du p-médian.....	5
1. Récupérer les coordonnées de nos différents points.....	5
2. Heuristiques.....	6
a) Heuristique gloutonne.....	6
b) Heuristique Random.....	7
c) Résolution par programme linéaire.....	8
3. Méta-heuristiques.....	9
a) Méta-heuristique gloutonne.....	9
b) Méta-heuristique random.....	9
III - Résolution du TSP.....	11
a) Heuristique du plus proche voisin.....	11
b) Méta-heuristique du plus proche voisin avec voisinage 2-OPT.....	11
IV - Résolution de l'anneau étoile.....	12
a) Résolution approchée.....	12
b) Résolution exacte.....	12
VI - Conclusion.....	14

Introduction

Ce rapport s'inscrit dans le cadre de la fin de notre U.E. Algorithmique et Optimisation, en deuxième année d'école d'ingénieur spécialité informatique. Dans le cadre de ce module, nous avons étudié trois matières principales : optimisation linéaire, optimisation combinatoire et complexité. Au cours de ces matières, nous avons étudié les bases de modélisations de problèmes par un programme linéaire et leurs résolutions par des méthodes exactes et approchées, comme le simplexe, des heuristiques gloutonnes et des améliorations de solutions par des métaheuristiques. Nous avons également pu étudier en complexité la différenciation connue entre les classes de problèmes P et NP et la preuve d'appartenance d'un problème à l'une de ces classes.

Nous avons donc, pour la fin de ce module, réalisé un projet d'optimisation visant à mettre en application ces notions à travers l'étude d'un problème de construction d'un réseau de transport public, modélisé par le problème de l'anneau étoile, un problème NP difficile combinant les deux sous problèmes du p-médian et du voyageur de commerce également NP-difficile pour former un cycle de p stations parmi des points donnés.

Nous verrons dans ce rapport comment nous avons pu résoudre un problème non polynomial en trouvant des solutions plus ou moins bonnes en des temps acceptables. Pour cela, nous montrerons d'abord l'appartenance du problème de l'anneau étoile à la classe NP, puis détaillerons les heuristiques et métaheuristiques utilisées pour résoudre ce problème. Nous comparerons leurs solutions trouvées ainsi que leur temps d'exécution afin de déterminer la pertinence de chacune d'entre elles.

I - Problème de l'anneau étoile, un problème NP-difficile.

Tout d'abord, montrons que le problème de l'anneau étoile est NP-difficile.

On va réaliser pour cela une réduction polynomiale depuis le problème du P-Médian que l'on sait NP-difficile.

Le problème du P-Médian vise à choisir p stations parmi n villes afin de minimiser la somme des distances entre chaque ville et sa station la plus proche.

Le problème de l'anneau étoile donné en énoncé vise, quant à lui, à choisir p stations parmi n villes et de tracer un cycle entre toutes les stations. On cherche ensuite à minimiser la somme de :

- (1) La somme entre chaque ville et sa station la plus proche
- (2) La longueur totale du cycle tracé entre les stations multiplié par un coefficient α .

On considère donc une instance du problème de l'anneau étoile où l'on définit $\alpha=0$.

Pour $\alpha=0$, la longueur totale du cycle entre les stations vaut 0 car on le multiplie par α .

Il nous reste donc l'objectif de minimiser le point (1) évoqué auparavant.

Or, minimiser ce point revient exactement à résoudre le problème du P-Médian car le coût du cycle est nul, seul le choix des p médians influence la valeur de la solution optimale.

On en conclut donc que le problème du p -médian est un cas particulier du problème de l'anneau étoile, le cas où $\alpha=0$.

Résoudre le problème général pour tout α de l'anneau étoile est donc au moins aussi difficile que de résoudre le problème du p -médian.

Enfin, puisque le problème du p -médian est connu NP-difficile, on en conclut que le problème de l'anneau étoile est également NP-difficile.

II - Résolution du p -médian

1. Récupérer les coordonnées de nos différents points

Le problème du p -médian vise à définir p stations parmi n points donnés et d'affecter chaque point n'étant pas une station à sa station la plus proche de manière à minimiser la somme des distances totales d'affectation. La première étape est donc de récupérer les n points sur lesquels on va travailler. Le sujet nous indique de travailler sur une instance réelle dédiée au problème du voyageur de commerce, et plus précisément, aux instances symétriques et données simplement par un nuage de points et leurs coordonnées.

Nous avons donc pris initialement le fichier "att48.tsp", qui correspond aux 48 capitales des États connexes des États-Unis.

Il nous suffit ensuite de récupérer chaque état avec ses coordonnées associées listées dans ce fichier. On extrait ces données avec la fonction "initCoordN()" dans le fichier "pMedian.jl" qui initialise le tableau "coords" avec des tuples (x, y) , où $coords[i]$ renvoie les coordonnées (x, y) de l'état i . Cette fonction calcule aussi les coordonnées des points extrêmes de notre nuage de point, ce qui nous sera utile par la suite pour l'heuristique gloutonne.

Au lancement du programme, on demande d'entrer le nom d'un fichier, pour utiliser l'instance des 48 États des États-Unis il vous suffit d'entrer att48.tsp. Il peut être intéressant d'essayer d'utiliser une instance du même format mais d'un autre pays avec différentes villes. Pour cela, il suffit juste de placer le fichier tsp de votre choix dans le répertoire "tspFile", de relancer le programme et d'entrer le nom de votre fichier à son lancement. Faites attention à bien rentrer le nom exact de votre fichier sinon le programme échoue avec l'erreur : ERROR: SystemError: opening file "../tspFile/'mauvaisNom'": Aucun fichier ou dossier de ce nom

Il aurait été intéressant de tester le programme sur un fichier tsp regroupant les villes françaises mais après quelques recherches internet, il paraît qu'il n'existe pas de fichiers tsp du même format que att48.tsp déjà construit. Nous avons donc récupéré le fichier "eil76.tsp" regroupant 76 points pour pouvoir tester le programme sur une autre instance.

Une fois que les états sont bien récupérés, on peut commencer l'implémentation de nos différentes heuristiques et métaheuristiques.

2. Heuristiques

a) Heuristique gloutonne

La première heuristique que nous avons implémentée pour définir p stations parmi nos n Etats récupérés précédemment est l'heuristique gloutonne. Cette heuristique fonctionne de la manière suivante :

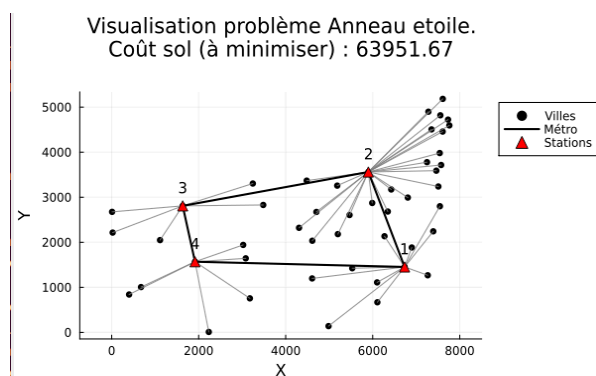
Nous récupérons les points extrêmes de notre instances et découpons l'espace des points en $q*q$ rectangles, avec $q = \lfloor \sqrt{p} \rfloor$. L'objectif étant de diviser l'espace des points en p rectangles où l'on va ensuite définir le point le plus proche du centre de chaque rectangle comme étant une station et auquel on va affecter tous les points non stations de ce même rectangle.

Cette heuristique peut donner de très bons comme de mauvais résultats, cela dépend de la répartition dans l'espace de nos points initiaux.

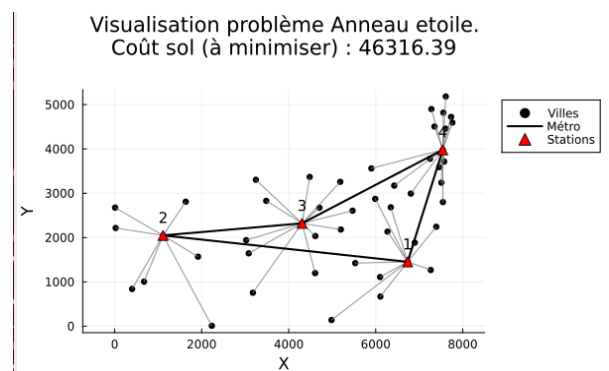
Si nos points sont bien répartis uniformément dans l'espace, alors la solution sera très bonne car en quadrillant la carte et en plaçant une station au centre de chaque zone, chaque État sera à une distance raisonnable d'une station, minimisant ainsi la somme des distances d'affectations.

Cependant, si les États ne sont pas répartis uniformément, en particulier si certains Etats sont regroupés à un endroit éloigné du centre d'un rectangle tandis qu'un état isolé se situe proche de ce centre, il sera alors sélectionné comme station mais l'ensemble des États de ce rectangle présenteront une distance d'affectation élevée, ce qui est défavorable pour le coût de la solution.

Ce problème s'illustre dans notre cas lorsque l'on veut par exemple définir 4 stations, comparons ce que l'on obtient en utilisant l'heuristique gloutonne la résolution par programme linéaire, donnant la solution optimale :



4 médians par heuristique gloutonne



4 médians par PL

On peut constater que de nombreuses stations sont situées au nord-est, le PL place donc une station au milieu de ces points. Ce n'est pas le cas de l'heuristique gloutonne, qui va définir le point 2 en station car étant au centre d'un rectangle sans prendre en compte le fait que de nombreuses stations sont regroupées au nord-est. Cela influe donc directement sur le coût de la solution qui est de 63 951.67m

pour l'heuristique gloutonne et de 46 316.39m pour le PL. On a choisi de définir $\alpha = 0$ afin de n'avoir que le coût du 4-médian sans compter le coût du TSP que l'on ne souhaite pas mesurer ici.

Un autre problème se pose avec cette heuristique, en effet, on a défini $q \times q$ rectangles, il y aura donc $q \times q$ stations définies avec cette méthode. Hors, prenons deux exemples, pour $p=7$ et $p=15$ on aura ceci :

Comme $q = \lceil \sqrt{p} \rceil$, on a :

si $p=7$, $\lceil \sqrt{7} \rceil = 3$ et $3 * 3 = 9$.

si $p=15$, 4 et $4 * 4 = 16$.

On aura donc dans un cas 9 stations au lieu de 7 et dans l'autre 16 stations au lieu de 15, nous devons donc définir deux méthodes corrigeant cela.

On a donc implémenté les deux fonctions "trierStations" pour le cas où nous avons trop de stations et "comblarStations" pour le cas inverse.

La fonction "trierStations" est assez intelligente. Tant qu'on a trop de stations, elle va chercher parmi nos stations définies les deux qui sont les plus proches et en supprimer une des deux afin d'éviter d'avoir des stations trop proches, ce qui n'est pas très utile.

La fonction "comblarStations" quant à elle est assez basique. Elle va créer une permutation aléatoire de tous les points dont nous disposons puis définir des stations parmi les points qui ne sont pas des stations. On aura donc un choix aléatoire des stations à rajouter pour combler le manque.

b) Heuristique Random

Afin d'avoir au moins deux heuristiques au choix pour définir les p médians, nous avons également implémenté une heuristique randomisée assez simple. On a la fonction "defStationsRandom" qui définit aléatoirement p stations parmi nos n points.

La solution de cette heuristique étant vraiment aléatoire et peu pertinente, on propose donc par défaut de la répéter un certains nombres de fois et gardant la meilleure solution trouvée. Le programme vous demande de définir le nombre d'essais à effectuer.

On aurait pu définir par défaut que cette heuristique lançait, par exemple, 100 itérations sans laisser le choix à l'utilisateur. Cependant, ce programme a été développé dans l'optique d'être destiné à des utilisateurs expérimentés et connaissant le sujet. On laisse donc de la flexibilité à l'utilisateur qui peut décider de chaque choix de résolution afin de pouvoir établir des comparaisons pertinentes entre les différentes méthodes de résolutions.

A titre de comparaison, avec l'instance att48.tsp, on a généralement avec 100 itérations de l'heuristique random une solution avec un coût équivalent ou plus faible, donc meilleur, qu'avec l'heuristique gloutonne.

On a comparé les heuristiques gloutonnes et random avec 100 itérations, combiné à la méthode de tracé de cycle du plus proche voisin pour avoir un meilleur temps d'exécution qu'avec la 2-OPT et $\alpha=0$.

Pour l'heuristique random, il est plus intéressant d'effectuer plusieurs exécutions de test et prendre une moyenne des coûts de solutions et des temps d'exécutions pour avoir une valeur plus représentative de cette heuristique car elle change à chaque exécution.

Nous avons donc implémenté la fonction "test_heuristiqueRandom100iter" dans le fichier "test.jl" et réalisé 15 exécutions de tests et récupérés leurs résultats moyens, inscrits dans le tableau suivant :

Heuristique p-médian	Heuristique TSP	Nombres de stations	Coût solution (m)	Temps d'exécution (ms)	Meilleure solution
Glouton / Random (100 iter)	Plus proche voisin	5	46849.42 / 43432.6	0.2 / 0.3	Random
//	/	15	21794.14 / 20821.97	0.2 / 1	Random
//	/	35	5307.77 / 5072.99	0.3 / 1.1	Random

Pour ce qui est des temps d'exécution, on ne les prend pas en compte pour définir la meilleure solution car ces deux méthodes sont très rapides. On a en moyenne pour placer 15 stations avec 100 itérations de l'heuristique aléatoire, une exécution en 0.001055722s, soit à peu près 1ms. Pour l'heuristique gloutonne, le programme s'exécute en 0.00024072s, soit environ 0.2ms. L'heuristique gloutonne est donc plus rapide mais ces temps d'exécutions restent négligeables donc on ne les prend pas en compte pour dire que l'heuristique Random est généralement meilleure que la gloutonne.

c) Résolution par programme linéaire

Enfin, une dernière manière de résoudre le problème du p-médian est la résolution d'un programme linéaire donné dans l'énoncé. La résolution de ce programme linéaire donne nécessairement la meilleure solution du p-médian, elle est donc très utile pour discuter de la pertinence d'une heuristique, il nous suffit de comparer le coût obtenu avec la résolution par PL à celui obtenu avec notre heuristique. Si les coûts sont proches, alors notre heuristique est très pertinente car elle est proche de la solution optimale, sinon, on peut dire qu'elle est peu efficace.

Vous remarquerez que nous avons déjà réalisé ce type de tests auparavant en comparant la résolution du 4-médian avec l'heuristique gloutonne et avec la résolution par PL pour discuter de l'efficacité de l'heuristique gloutonne.

3. Méta-heuristiques

a) Méta-heuristique gloutonne

Une méta-heuristique permet d'améliorer une solution heuristique afin d'avoir de meilleurs résultats. Afin d'améliorer notre heuristique gloutonne, on a utilisé la méthode suivante :

On génère une solution du p-médian par l'heuristique gloutonne, puis on va essayer un swap entre un point qui est station et un point qui n'est pas station. Après cet échange, on calcule le nouveau coût de notre solution et si il est inférieur au coût avant le swap, on garde alors cette nouvelle distribution de stations.

Évidemment, on ne va pas essayer qu'un seul swap, notre objectif serait d'essayer d'atteindre un minimum global de notre coût. Pour cela, on a implémenté la fonction "applicationStochastique", qui va définir arbitrairement un compteur égal à $2*n$, avec n notre nombre de points total.

On va ensuite décrémenter le compteur à chaque échange de station n'améliorant pas la solution. Chaque fois que la solution est améliorée, on remet le compteur à $2*n$. Notre algorithme s'arrêtera donc uniquement lorsque l'on aura fait $2*n$ échanges de stations sans aucune amélioration.

Avec l'heuristique gloutonne, pour $p=7$ et $\alpha=1$ et la méthode de tracé de cycle du plus proche voisin 2-opt, on trouve une solution avec un coût égal à 54424.25m en 0.2ms.

Initialement, le compteur était initialisé et remis à n à chaque améliorations, mais après quelques tests, on voit pour $p=7$, que l'on obtient un coût de 53051.6m en 0.7ms avec cette méta-heuristique, une amélioration de 2.52% par rapport à l'heuristique gloutonne, l'amélioration est donc assez faible pour un p petit. On a donc choisi de placer le compteur à $2*n$, on obtient un coût de 49082.87m en 1ms, soit une amélioration de 9.81% par rapport à l'heuristique gloutonne et en un temps d'exécution toujours très rapide, on garde donc cette initialisation du compteur.

En le plaçant à $3*n$, on obtient un coût de 49053.28m, il n'y a pas de grandes améliorations, il est donc inutile de rajouter du temps de calcul. Evidemment, cela est vrai pour notre instance de att48.tsp mais cela dépend directement du nombre de points en entrée, néanmoins, on peut prendre cette instance pour exemple et généraliser pour les autres, l'avantage de coût en plaçant le compteur à $3*n$ ne devrait pas avoir un gain très important.

b) Méta-heuristique random

Ensuite, pour améliorer l'heuristique random, nous avons procédé de la même manière que pour la méta-heuristique gloutonne. La seule différence réside dans le nombre d'itérations réalisées. En effet, l'heuristique gloutonne pour du p -médian donne toujours le même résultat pour un même p donné, contrairement à l'heuristique random.

On va donc demander à l'utilisateur de rentrer le nombre d'essais à effectuer et pour chaque essai, on va générer une heuristique random sur laquelle on va appliquer l'algorithme "applicationStochastique" vu précédemment. On a donc généré nbEssais heuristiques random que l'on a améliorée avec "applicationStochastique", parmi toutes ces solutions, on garde finalement la meilleure trouvée qui devrait fournir un très bon résultat.

Voici un tableau de comparaisons entre une heuristique et sa méta heuristique pour chaque méthode. Les résultats des méta-heuristiques gloutonnes, aléatoire ainsi que de l'heuristique aléatoire sont des moyennes de 15 tests effectués avec $p=10$, $\alpha=0$ et résolution du tsp par l'heuristique de plus proche voisin :

Heuristique et métaheuristique	Coût heuristique / méthaheuristique (m)	Temps heuristique / méthaheuristique (ms)	Amélioration (%)
Gloutonne vs métaGloutonne	28548.63 / 25826.48	0.2 / 0.9	9.5
Aléatoire (100 iter) vs métaAléatoire	28412.05 / 24922.55	0.7 / 53	12.3

Pour la méta-heuristique aléatoire, l'exécution reste toujours très rapide mais prend tout de même un temps 50 fois supérieur à l'heuristique aléatoire pour s'exécuter, si on essaie de faire trop d'itérations de la méta-heuristique aléatoire, le programme pourrait prendre beaucoup de temps à s'exécuter.

On remarque cependant, en utilisant la résolution par PL pour $p=10$ et $\alpha=0$ que le programme trouve un coût de 24917.94m en 50ms, soit presque la même solution que celle trouvée par la méta-heuristique aléatoire. On a donc une méta-heuristique qui donne une solution très proche de la

solutions optimale mais qui met presque le même temps à s'exécuter que le programme linéaire, ce qui n'est donc pas très intéressant.

Si on augmente p , pour $p=20$, on a :

Avec la méta-heuristique aléatoire sur 100 itérations : coût moyen = 12377.11m, temps moyen = 96ms

Avec la méta-heuristique gloutonne : coût moyen = 12932.99m, temps moyen = 1.2ms

Avec le programme linéaire : coût = 12366.34m, temps = 34ms

La solution la plus proche de celle obtenue avec le programme linéaire est encore la méta-heuristique aléatoire. Cependant, elle est longue à s'exécuter, plus longue que le PL. La méta-heuristique gloutonne quant à elle s'exécute bien plus rapidement, on peut donc préférer utiliser la méta-heuristique gloutonne pour avoir une bonne solution très rapidement.

Le principal défaut de nos méta-heuristique est qu'elles ne prennent en compte que la résolution du p -médian, sur lequel on va ensuite superposer une solution du TSP et renvoyer le coût total. Elles ne calculent que séparément les problèmes du p -médian puis du TSP pour trouver une solution du problème de l'anneau étoile.

Une amélioration possible aurait été de calculer le cycle du TSP à chaque swap de station dans la fonction "applicationStochastique", on aurait alors pu additionner les coûts du p -médian et du tsp et voir si on obtient une amélioration.

Nos deux mét-heuristiques, gloutonne et aléatoire, auraient alors été plus performantes globalement. Cependant, recalculer un cycle de TSP avec l'heuristique plus proche voisin ou pire, plus proche voisin 2opt pour chaque swap de station génère beaucoup de calculs, ce qui aurait grandement ralenti nos méta-heuristiques, en particulier l'aléatoire qui prend déjà un peu de temps à s'exécuter. On a donc fait le choix de ne pas les modifier, car elles ont déjà d'assez bonnes performances.

III - Résolution du TSP

a) Heuristique du plus proche voisin

On passe maintenant à la résolution du problème du voyageur de commerce (Traveling Salesman Problem). Pour ce faire, on a d'abord implémenté l'heuristique du plus proche voisin. Le sujet impose que le point 1 est toujours une station, on va donc partir de ce premier point aller à la station la plus proche et ainsi de suite. Une fois arrivé à la dernière station, on retourne à la première. On finira donc bien par visiter toutes les stations une unique fois et former un cycle, comme le demande le problème du voyageur de commerce.

Cette heuristique est implémentée dans le fichier "[tsp.jl](#)" à l'aide de la fonction "plusProcheVoisin". Elle est cependant assez loin de la solution optimale car elle ne va qu'à la station la plus proche par rapport à notre position actuelle, elle peut donc créer des croisements d'arêtes qui augmentent la longueur totale du cycle, que l'on cherche à minimiser.

b) Méta-heuristique du plus proche voisin avec voisinage 2-OPT

Pour améliorer cette heuristique, on va donc améliorer la solution par une méthode itérative mettant en œuvre le voisinage 2-OPT. Le voisinage 2-OPT à partir d'un point désigne l'ensemble des solutions obtenues en supprimant deux arêtes non adjacentes du cycle existant et en les reconnectant de la seule autre manière possible pour former un nouveau cycle sans croisements d'arêtes.

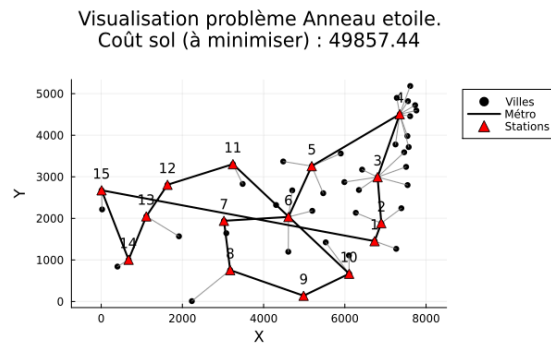
Cette méta-heuristique est implémentée dans la fonction "plusProcheVoisin2Opt", génère une solution initiale avec l'algorithme plusProcheVoisin et l'améliore en supprimant les croisements.

Cette approche permet d'obtenir des cycles plus courts et de se rapprocher de la solution optimale pour le TSP.

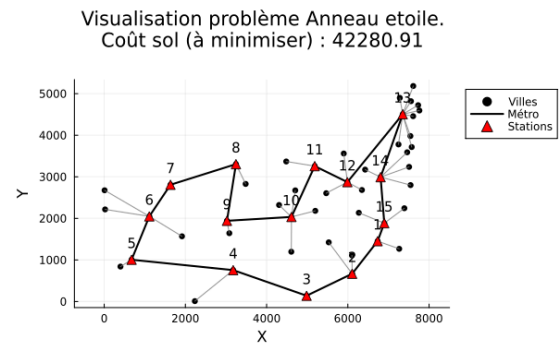
On réalise un test pour $p=15$ et $\alpha=1$ avec résolution du 15-médian par l'heuristique gloutonne avec ces deux méthodes et voici les résultats :

Avec l'heuristique du plus proche voisin : coût = 49857.44m, temps = 0.24ms

Avec l'heuristique du plus proche voisin 2-OPT : coût = 42280.91m, temps = 0.26ms



15-médians avec ppv



15-médians avec ppv 2-OPT

Graphiquement, on voit bien que l'algorithme du plus proche voisin crée des croisements entre les arêtes contrairement à la méta-heuristique avec le voisinage 2-OPT.

Pour cette solution, l'heuristique gloutonne donne toujours la même solution, de même que l'heuristique plus proche voisin et la méta-heuristique plus proche voisin 2-OPT. Il est donc inutile de répéter ces tests plusieurs fois et de faire une moyenne des résultats.

IV - Résolution de l'anneau étoile

a) Résolution approchée

Finalement, la résolution du problème de l'anneau étoile réside donc finalement à la superposition d'une résolution du p-médian ainsi que d'une résolution du TSP comme on a pu le voir précédemment.

Une résolution approchée du problème revient donc à effectuer n'importe quelle des méthodes vues jusqu'ici, en utilisant soit une heuristique, soit une méta-heuristique pour les problèmes du p-médian et du TSP.

Vous remarquerez qu'une résolution par programme linéaire du p-médian correspond également à une heuristique du problème de l'anneau étoile.

En effet, le programme linéaire du p-médian ne minimise que les coûts des affectations des points non médians aux points médians sans prendre en compte la longueur du cycle du TSP. On le combine ensuite avec une heuristique ou méta-heuristique du TSP, cela ne peut donc pas définir une définition exacte.

Pour trouver la solution optimale du problème de l'anneau étoile, il va donc falloir implémenter un nouveau programme linéaire avec pour objectif de minimiser simultanément les affectations des non médians et la longueur du cycle pondéré par le coefficient "alpha" formé par les points médians.

b) Résolution exacte

Le PLNE compacte est donné dans l'énoncé, il modélise le problème à l'aide de 3 variables :

- Les variables y_{ij} , qui déterminent les points qui sont stations ainsi que leur affectation. Si $y_{ii}=1$, alors i est une station et si $y_{ij}=1$, alors le non médian i est affecté au médian j . Ces variables sont similaires à celles utilisées pour le plne du p -médian.
- Les variables x_{ij} , qui représentent les arêtes du cycle. Si $x_{ij}=1$, alors les médians i et j sont des médians reliés par une arête.
- Les variables z_{ij} , des variables dites de flots, elles permettent de forcer les arrêtes à former un cycle passant par le sommet 1 et d'exactement p arrêtes.

L'implémentation de ce programme linéaire se situe à la fin du fichier "[anneauEtoile.jl](#)", avec les fonctions "ae_plne", "uneStation" et "resolution".

La fonction "ae_plne" va définir le modèle, les variables ainsi que les contraintes du problème. Par ailleurs, il va falloir définir deux contraintes supplémentaires que celle explicitées par le PLNE, une contrainte forçant le point 1 à être une station ainsi qu'une contrainte interdisant les boucles d'une arête sur un même sommet. On a commenté le code, les définissant comme étant les contraintes (8) et (9).

La fonction "uneStation" permet de gérer le cas limite $p=1$ qui provoquait une erreur car le solveur ne pouvait pas faire de cycle avec un seul point.

Enfin, la fonction "resolution" remplit les tableaux stations, affect et ordreDeVisite grâce aux valeurs des variables x , y et z trouvées par le PLNE.

La résolution de ce PL nous donne la garantie d'avoir la solution optimale pour le problème de l'anneau étoile. Cependant, cette formulation est très complexe, elle combine les complexités des problèmes du p -médian et du TSP qui sont tous deux NP-difficile.

Elle utilise des variables binaires pour chaque paire de villes, x_{ij} pour le cycle et y_{ij} pour l'affectation. En fonction du nombre de points n donnés en entrée, cela peut nous donner énormément de variables et de contraintes, il y aurait, rien que pour les variables x et y : $n*n + n*n$ variables et contraintes, soit $2*n^2$ variables et contraintes en plus des variables z et de ses contraintes associées.

Avec le fichier "att48.tsp", donc pour $n=48$, le solveur GLPK peut résoudre ce problème en un temps raisonnable pour un nombre de stations à définir allant jusqu'à 4. On définit donc une durée maximale de résolution à 2 minutes, si au bout de 2 minutes, le solveur n'a pas trouvé la solution optimale, il renvoie la meilleure solution qu'il a trouvé jusqu'à maintenant.

Pour $p=4$, on trouve la solution optimale en 12.8s, le coût est de 59932.7m.

Pour $p=5$, le solveur ne trouve pas la solution optimale en 2 minutes, il nous renvoie alors la meilleure solution trouvée : coût = 55444.97m

Si on compare cette résolution à nos nos méta-heuristiques, on trouve les moyennes de coûts et de temps d'exécutions suivants :

Avec la méta-heuristique gloutonne : coutMoyen = 57706.58, tempsMoyen = 0.5ms

Avec la méta-heuristique aléatoire sur 100 itérations : coutMoyen = 56832.71, tempsMoyen = 9ms

On trouve donc en moyenne des solutions proches de la solution optimale en un temps très rapide, ce qui prouve que nos méta-heuristiques sont efficaces.

Grâce à notre compte étudiant, on a pu installer le solveur CPLEX gratuitement, un solveur bien plus efficace que GLPK.

Cependant même avec ce solveur, les temps de calculs sont assez long. On a testé pour $p=5$ puis pour $p=10$ et voici nos résultats :

Pour $p=5$, on trouve la solution optimale en 7.8s donc bien plus rapide que GLPK, on a alors le coût minimal : coût = 55316.25.

On voit donc que GLPK ainsi que nos méta-heuristiques étaient très proche de la solution optimale.

On essaye enfin pour $p=10$, $\alpha=1$ et comparons chacune des méthodes :

Méthode de résolution	Coût (m)	Temps (ms)
PL (Cplex)	42269.87	$1796.76 * 10^3$
PL (GLPK)	44969.81	$1860.75 * 10^3$
Méta-heuristique gloutonne	47577.8	0.5
Méta-heuristique aléatoire (100 iter)	47257.75	50

On note que l'on a laissé tourné CPLEX sans limite de temps, sachant qu'il finirait par trouver la solution optimale, il a donc pu tourner pendant 29.9 minutes.

Pour GLPK, on a donc fixé une limite de temps à 31 minutes pour avoir une solution comparable à celle obtenue avec CPLEX.

Vous pouvez voir les résultats de vos exécutions dans le fichier historique.txt qui s'ouvre au lancement du programme.

VI - Conclusion

Ce projet nous a permis d'explorer la résolution d'un problème NP-difficile combinant deux sous-problèmes NP-difficile.

La résolution exacte de ce problème se heurte assez vite à un problème de temps d'exécution pour $p > 4$ en fonction du solveur choisi, ce qui nous a obligé à implémenter nos heuristiques et méta-heuristiques présentées dans ce rapport.

Si l'on veut résoudre efficacement ce problème, il est nécessaire de passer par une des méta-heuristique, aléatoire ou gloutonne avec un tracé de cycle suivant l'heuristique du plus court chemin amélioré par un voisinage 2-OPT, ce qui, comme on l'a vu, peut donner une solution très proche de la solution optimale en un temps inférieur à une seconde, qui représente un gain de temps considérable par rapport à la résolution exacte.

Pour conclure, on peut dire que parvenir à résoudre ce problème et optimiser les temps de calculs via différentes méthodes était très instructifs et a permis de bien mettre en pratique les notions théoriques vu lors de notre module Algorithmique et Optimisation.